Embedded C Programs and Output Screenshots

1. Temperature Sensor using ADC and Cooling Fan Control

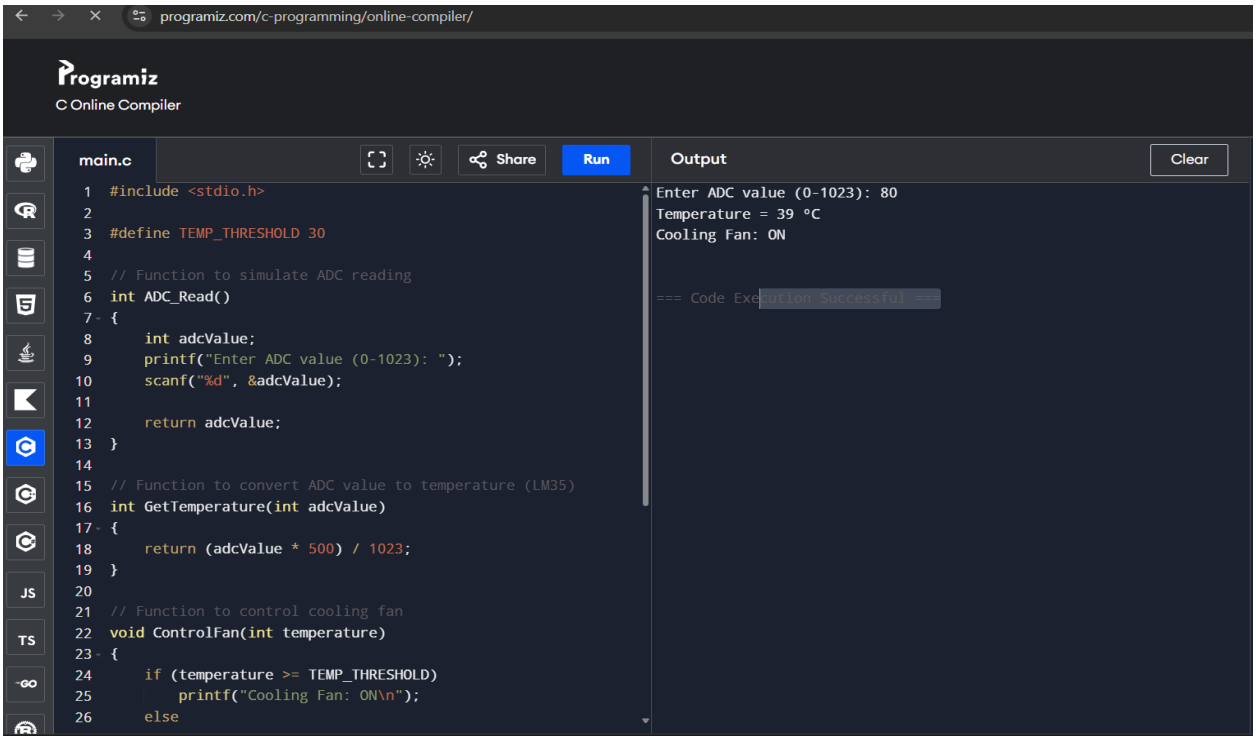
```
#include <stdio.h>

#define TEMP_THRESHOLD 30

int ADC_Read()
{
    int adcValue;
    printf("Enter ADC value (0-1023): ");
    scanf("%d", &adcValue);
    return adcValue;
}

int GetTemperature(int adcValue)
{
    return (adcValue * 500) / 1023;
}

void ControlFan(int temperature)
{
    if (temperature >= TEMP_THRESHOLD)
        printf("Cooling Fan: ON\n");
    else
        printf("Cooling Fan: OFF\n");
}

int main()
{
    int adcValue = ADC_Read();
    int temperature = GetTemperature(adcValue);

    printf("Temperature = %d °C\n", temperature);
    ControlFan(temperature);

    return 0;
}
```

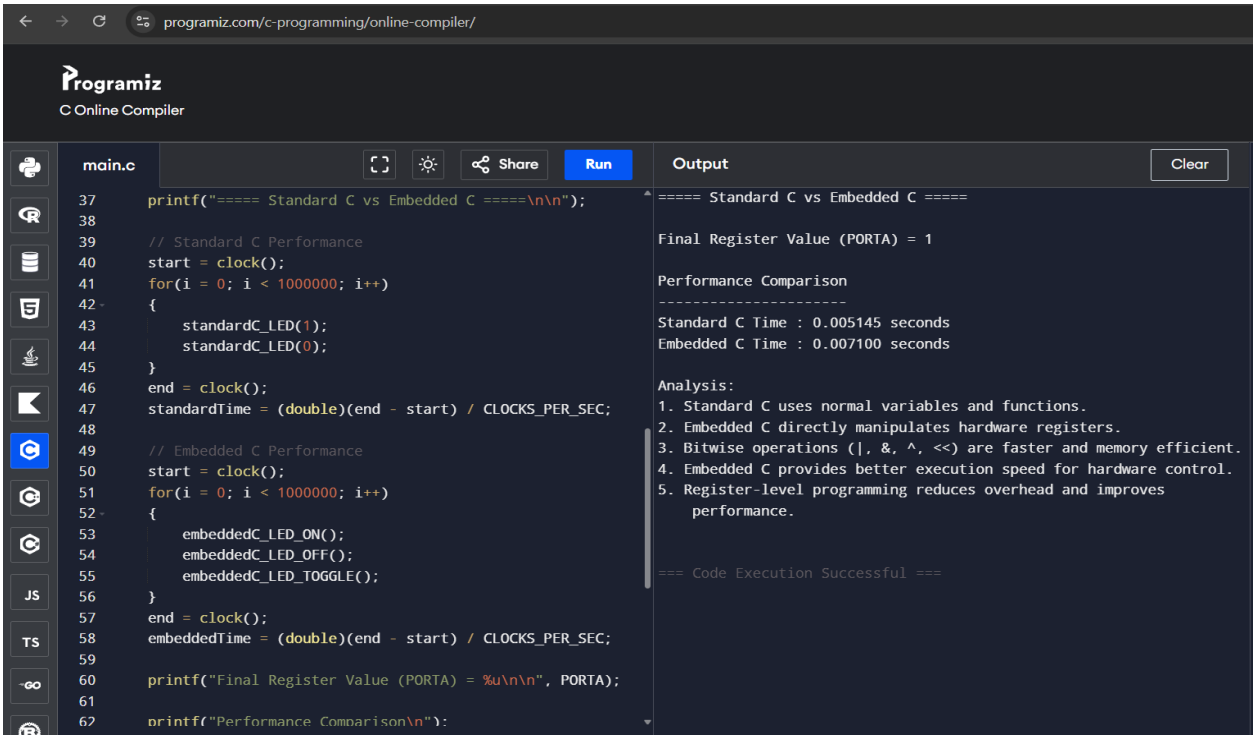
2. Standard C vs Embedded C Comparison

```
#include <stdio.h>
#include <time.h>

volatile unsigned char PORTA = 0x00;

void standardC_LED(int state)
{
    if(state) PORTA = 1;
    else PORTA = 0;
}

void embeddedC_LED_ON(){ PORTA |= (1<<0); }
void embeddedC_LED_OFF(){ PORTA &= ~(1<<0); }
void embeddedC_LED_TOGGLE(){ PORTA ^= (1<<0); }

int main()
{
    int i;
    clock_t start,end;
    double standardTime,embeddedTime;

    start=clock();
    for(i=0;i<1000000;i++){
        standardC_LED(1);
        standardC_LED(0);
    }
    end=clock();
    standardTime=(double)(end-start)/CLOCKS_PER_SEC;

    start=clock();
```

```
    for(i=0;i<1000000;i++){
        embeddedC_LED_ON();
        embeddedC_LED_OFF();
        embeddedC_LED_TOGGLE();
    }
    end=clock();
    embeddedTime=(double)(end-start)/CLOCKS_PER_SEC;

    printf("PORTA=%u\n",PORTA);
    printf("Standard C Time=%f\n",standardTime);
    printf("Embedded C Time=%f\n",embeddedTime);
    return 0;
}
```

Output Screenshots




programiz.com/c-programming/online-compiler/
Programiz
C Online Compiler
main.c
Share
Run
Output
Clear
printf("===== Standard C vs Embedded C =====\n\n");
// Standard C Performance
start = clock();
for(i = 0; i < 1000000; i++)
{
standardC_LED(1);
standardC_LED(0);
}
end = clock();
standardTime = (double)(end - start) / CLOCKS_PER_SEC;
// Embedded C Performance
start = clock();
for(i = 0; i < 1000000; i++)
{
embeddedC_LED_ON();
embeddedC_LED_OFF();
embeddedC_LED_TOGGLE();
}
end = clock();
embeddedTime = (double)(end - start) / CLOCKS_PER_SEC;
printf("Final Register Value (PORTA) = %u\n\n", PORTA);
printf("Performance Comparison\n");
===== Standard C vs Embedded C =====
Final Register Value (PORTA) = 1
Performance Comparison
Standard C Time : 0.005145 seconds
Embedded C Time : 0.007100 seconds
Analysis:
1. Standard C uses normal variables and functions.
2. Embedded C directly manipulates hardware registers.
3. Bitwise operations (|, &, ^, <<) are faster and memory efficient.
4. Embedded C provides better execution speed for hardware control.
5. Register-level programming reduces overhead and improves
performance.
=== Code Execution Successful ===